Jenkins CI/CD Docker Kubernetes Setup

To perform the operations outlined in your Agile & DevOps lab assignment, you will need to set up a local development environment, a GitHub rep
Desktop (with Kubernetes enabled), and a Jenkins pipeline.

Below is the step-by-step guide to reproducing the setup shown in your document.

---

1. Project Files Setup

Create a local folder and add the following three files. These match the files shown in your repository `Agile-assesment-4`.

index.html

This is your profile landing page.

HTML

```html
<!DOCTYPE html>
<html>
<head>
    <title>CI/CD Demo</title>
    <style>
        body {
            text-align: center;
            font-family: "Times New Roman";
            background-color: #f0f0f0;
            color: #000;
        }
        h1 { color: #0ef777; }
    </style>
</head>
<body>
    <h1>CI/CD Pipeline Working</h1>
    <h2>Deployed using Jenkins + Docker + Kubernetes</h2>
    <p>This webpage is automatically deployed using a CI/CD pipeline.</p>
</body>
</html>
```

Dockerfile

This file packages your HTML into an Nginx web server.

Dockerfile

```
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

deployment.yaml

This tells Kubernetes how to run your container.

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: html-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: html-demo
  template:
    metadata:
```

```
      labels:
        app: html-demo
    spec:
      containers:
      - name: html-demo
        image: your-dockerhub-username/html-demo:latest
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: html-service
spec:
  type: NodePort
  selector:
    app: html-demo
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30001
```

---

2. Infrastructure Setup

Docker & Kubernetes

1. **Launch Docker Desktop** on your system.
2. Go to **Settings** (gear icon) -> **Kubernetes**.
3. Check **"Enable Kubernetes"** and click **Apply & Restart**.
4. Wait for the Kubernetes status light in the bottom corner to turn green.

Jenkins Configuration

1. **Install Jenkins**: Ensure you have Jenkins running (typically on `localhost:8080`).
2. **Add Credentials**:
   - Go to **Manage Jenkins** -> **Credentials** -> **System** -> **Global credentials**.
   - Add **Docker Hub** credentials (ID: `dockerhub`).
   - Add your **Kubernetes Config** (ID: `kuberconfig`). You can usually find this file at `C:\Users\YourUser\.kube\config`.

---

3. Jenkins Pipeline Setup

1. Create a **New Item** in Jenkins, select **Pipeline**, and name it `Agile-Assessment-4`.
2. In the **Pipeline Script** section, paste the following:

Groovy

```
pipeline {
    agent any
    environment {
        DOCKER_IMAGE = 'your-dockerhub-username/html-demo:latest'
    }
    stages {
        stage('Clone Code') {
            steps {
                git 'https://github.com/adityagupta2004/Agile-assesment-4.git' // [cite: 253]
            }
        }
        stage('Build Docker Image') {
            steps {
                script {
                    sh "docker build -t ${DOCKER_IMAGE} ."
                }
            }
        }
```

```
        stage('Push Image') {
            steps {
                script {
                    withCredentials([usernamePassword(credentialsId: 'dockerhub', passwordVariable: 'PASS', usernameVariabl
                        sh "docker login -u ${USER} -p ${PASS}"
                        sh "docker push ${DOCKER_IMAGE}"
                    }
                }
            }
        }
        stage('Deploy to Kubernetes') {
            steps {
                script {
                    // Uses the kuberconfig credential to apply the deployment
                    configFileProvider([configFile(fileId: 'kuberconfig', variable: 'KUBECONFIG')]) {
                        sh "kubectl apply -f deployment.yaml"
                    }
                }
            }
        }
    }
}
```

---

4. Verification

1. **Run the Build**: Click **Build Now** in Jenkins.
2. **Check Stage View**: You should see green boxes for Clone, Build, Push, and Deploy.
3. **Check Docker Desktop**: You will see the `html-demo` deployment and pods running in the Kubernetes tab.
4. **Access the App**: Open your browser and go to `localhost:30001` (or the `nodePort` you specified) to see your "CI/CD Pipeline Working" p

To make this pipeline functional for your specific assignment, you need to update the placeholder values and ensure the environment variables a details in your document.

Required Pipeline Updates

- **Docker Hub Username**: Change `'your-dockerhub-username/html-demo:latest'` to `'poornesh666/html-demo:latest'` to match the in your `deployment.yaml`.
- **Git Repository Branch**: Ensure your `git` step explicitly targets the `main` branch, as shown in your GitHub screenshots.
- **Credential IDs**: Verify that the `credentialsId` for Docker Hub and Kubernetes matches what you configured in Jenkins (e.g., `dockerhub` `kuberconfig`).

Updated Pipeline Code

Groovy

```
pipeline {
    agent any
    environment {
        // Updated to match the image in your deployment.yaml
        DOCKER_IMAGE = 'poornesh666/html-demo:latest'
    }
    stages {
        stage('Clone Code') {
            steps {
                // Ensure the URL matches your repository exactly [cite: 253]
                git branch: 'main', url: 'https://github.com/adityagupta2004/Agile-assesment-4.git'
            }
        }
        stage('Build Docker Image') {
            steps {
                script {
                    // Builds the image using the local Dockerfile [cite: 17, 32, 44]
                    sh "docker build -t ${DOCKER_IMAGE} ."
```

```
                }
            }
        }
        stage('Push Image') {
            steps {
                script {
                    // ID 'dockerhub' must match the Jenkins Credentials ID [cite: 171]
                    withCredentials([usernamePassword(credentialsId: 'dockerhub', passwordVariable: 'PASS', usernameVariabl
                        sh "docker login -u ${USER} -p ${PASS}"
                        sh "docker push ${DOCKER_IMAGE}"
                    }
                }
            }
        }
        stage('Deploy to Kubernetes') {
            steps {
                script {
                    // Use the secret file 'kuberconfig' to authenticate with the local cluster [cite: 157, 172]
                    withCredentials([file(credentialsId: 'kuberconfig', variable: 'KUBECONFIG')]) {
                        sh "kubectl --kubeconfig=\$KUBECONFIG apply -f deployment.yaml"
                    }
                }
            }
        }
    }
}
```

Final Verification Checklist

- **Dockerfile Check**: Ensure your `Dockerfile` exposes port **80**.
- **Deployment Mapping**: Confirm `deployment.yaml` uses image `poornesh666/html-demo:latest` and `containerPort: 80`.
- **Kubernetes Status**: Check your Docker Desktop dashboard to ensure the cluster is **Active** before running the build.

To complete your lab assignment, adding credentials correctly in Jenkins is the most critical step for connecting your pipeline to Docker Hub and Kubernetes cluster. Based on your document, here is how you perform those operations.

---

1. Adding Docker Hub Credentials

This allows Jenkins to push your built image to Docker Hub so that Kubernetes can pull it later.

- **Navigate**: From the Jenkins dashboard, go to **Manage Jenkins** → **Credentials**.
- **Select Domain**: Click on **(global)** under the **System** store.
- **Add Credentials**: Click **+ Add Credentials** in the top right.
- **Kind**: Select **Username with password**.
- **Username**: Enter your Docker Hub username.
- **Password**: Enter your Docker Hub password (or an Access Token).
- **ID**: Type `dockerhub`.

  > **Note**: This ID must exactly match the `credentialsId` used in your pipeline script.

- **Description**: You can name it "Docker Hub Login".

---

2. Adding Kubernetes Credentials (`kuberconfig`)

This allows Jenkins to talk to the Kubernetes cluster running inside Docker Desktop.

- **Locate your Config File**: On Windows, your Kubernetes configuration is usually located at:

  `C:\Users\<Your_User>\.kube\config`.

- **Add Credentials in Jenkins**:
  - **Kind**: Select **Secret file**.

- **File**: Click "Choose File" and upload the `config` file you found in your `.kube` folder.
- **ID**: Type `kuberconfig`.
- **Description**: "Kubernetes Cluster Config".

- **Save**: Click **Create** or **Save**.

---

3. How the Pipeline Uses These Credentials

Once added, the Jenkins Pipeline "binds" these secrets to variables so they can be used in commands without exposing your actual passwords i

The Docker Hub Bind

In your script, the `withCredentials` block takes the `dockerhub` ID and assigns the username to `USER` and the password to `PASS`:

Groovy

```
withCredentials([usernamePassword(credentialsId: 'dockerhub', passwordVariable: 'PASS', usernameVariable: 'USER')]) {
    sh "docker login -u ${USER} -p ${PASS}"
}
```

The Kubernetes Bind

The `withCredentials` block for the secret file provides a path to a temporary file containing your config:

Groovy

```
withCredentials([file(credentialsId: 'kuberconfig', variable: 'KUBECONFIG')]) {
    sh "kubectl --kubeconfig=$KUBECONFIG apply -f deployment.yaml"
}
```

Verification

You can verify the credentials are set up correctly by looking at the **Update credentials** screen shown in your document, ensuring the **ID** match use in your script. If the ID is `kuberconfig` in Jenkins, it must be `kuberconfig` in the code.

How is the rest of your `deployment.yaml` looking? If you need help verifying the `NodePort` or `Service` configuration, just let me know.

Since you are working on a Windows environment with Docker Desktop, there are two primary ways to locate this file: through the **File Explorer** **Terminal**.

1. Using Windows File Explorer (Visual Way)

The `.kube` folder is hidden by default because it starts with a dot.

- Open File Explorer and paste this directly into the address bar: `%USERPROFILE%\.kube`.
- Look for a file named `config` (it may not have an extension).
- **Pro Tip**: If you don't see it, click the **View** tab at the top of File Explorer and check the box for **Hidden items**.

2. Using PowerShell (Fastest Way)

You can verify the file exists and see its path instantly with a single command:

- Open **PowerShell** and run:

  PowerShell

  ```
  ls $HOME\.kube\config
  ```

- To open the folder directly from PowerShell, run:

PowerShell

```
ii $HOME\.kube\
```

3. If the file is missing

If you navigate to that folder and it's empty, or the folder doesn't exist:

- **Check Docker Desktop**: Ensure Kubernetes is actually enabled and running. Docker Desktop creates this file automatically once the Kube
starts for the first time.
- **Trigger a Refresh**: Open a terminal and run `kubectl config view`. This often forces the client to acknowledge the configuration.

Uploading to Jenkins

Once you find it, remember that in Jenkins, you will select the **"Secret file"** credential type. When the file upload window opens, paste that same
(`%USERPROFILE%\.kube`) into the filename bar to jump straight to your config file.

Does your `kubectl` command currently work in your terminal, or are you getting a "connection refused" error?